

DBMS COURSE PROJECT REPORT

TRAINING INSTITUTE MANAGEMENT SYSTEM



Submitted by	MANEHEYA
Roll Number	25WU0102149
Course / Section	B.Tech - AIML/Panthers]
Faculty Name	Dr.kiran Mayee Adavala
Academic Year	2026-2027

**Department of Artificial Intelligence and Machine Learning
WOXSEN UNIVERSITY**

1 Abstract

The Training Institute Management System is a database-driven application developed to organize and manage the core operations of a training institute. The project stores structured information about courses, modules, sessions, trainers, batches, learners, enrollments, attendance, assessments, fee payments and certificates. A MySQL database is used for persistent storage, while a Python Tkinter graphical user interface provides a simple way to view, insert and delete records. The system demonstrates practical database concepts such as table design, primary keys, data manipulation, query execution and application-to-database connectivity. The user interface is intentionally focused on the live operations required for the final DBMS demonstration: viewing records, inserting new records and deleting records with the corresponding change reflected in MySQL. .

2. Introduction

Training institutes handle many related data items every day: available courses, modules under each course, scheduled sessions, trainers, learner details, batch allocation, attendance, marks, fees and certificate eligibility. Managing these records manually or in separate files makes it difficult to keep information consistent and retrieve it quickly.

3. Problem Statement

The problem addressed by this project is the need for a centralized database system that can store and organize training institute information in a structured form. The system should allow the user to access current records and perform basic database operations through an easy-to-use interface instead of entering every SQL command manually. The application therefore connects a Tkinter user interface to a MySQL database and supports record viewing, insertion and deletion.

4. Objectives and Scope

4.1 Objectives

- Design a structured relational database for a training institute.
- Store information about courses, modules, sessions, trainers, batches and learners.
- Maintain learner enrollments, attendance, assessment scores, fee records and certificate details.
- Provide a graphical interface for viewing, inserting and deleting database records.
- Connect Python to MySQL using mysql-connector-python.
- Demonstrate that UI operations are reflected directly in the MySQL database.
- Use primary keys and appropriate data types to identify and store records accurately.

4.2 Scope

The current implementation is intended as a DBMS course project and covers administrative record management rather than a complete commercial learning management platform. It supports 11 tables and the three live UI operations required for the project demonstration. Functions such as user login, advanced search, role-based access, automated certificate generation and online payments are outside the present scope and are listed as possible future enhancements.

5. Software and Hardware Requirements

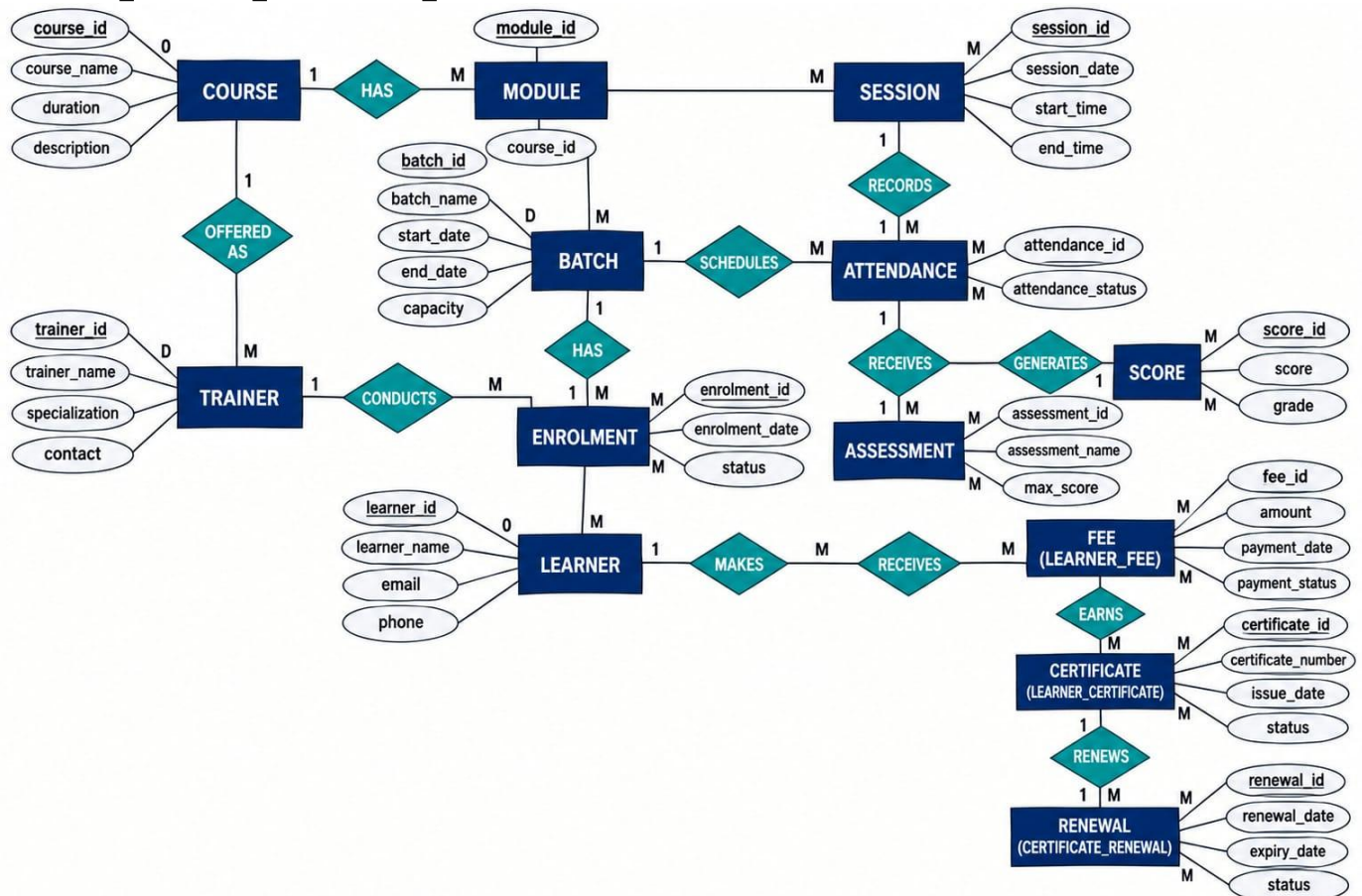
Category	Requirement
OperatingSystem	Windows 10/11
Programming Language	Python 3.x
Database	MySQL Server
Database Tool	MySQL Workbench
Database Connector	mysql-connector-python
Editor / IDE	Visual Studio Code or any Python editor
Version Control	GitHub

5.1 Minimum Hardware

- Processor: Dual-core processor or better
- RAM: 4 GB minimum
- Storage: At least 500 MB free for tools and project files
- Display: Standard laptop/desktop display
- Internet: Required for package installation and GitHub upload

6. ER Diagram

The ER diagram should represent the main entities and their logical relationships. The supplied SQL script defines the tables and primary keys but does not declare SQL FOREIGN KEY constraints. Therefore, the diagram should show the intended relationships based on matching identifier columns such as course_id, trainer_id, learner_id, batch_id, enrolment_id, module_id and session_id.



6.1 Main Logical Relationships

- Course -> Module: one course can contain multiple modules.
- Course -> Batch: one course can be offered through one or more batches.
- Trainer -> Batch: each batch stores the trainer assigned to it.
- Module -> Session: a module can have multiple teaching sessions.
- Learner -> Enrollment: a learner is registered through an enrollment record.
- Batch -> Enrollment: a batch can contain multiple learner enrollments.
- Enrollment -> Attendance / Assessment / Fees / Certificate: learner activity and outcome records are associated with an enrollment.

7. Relational Schema and Normalization

7.1 Relational Schema

Course	course_id (PK), course_name, duration_weeks, fee
Module	module_id (PK), course_id, module_name
Trainer	trainer_id (PK), trainer_name, specialization, phone
Session	session_id (PK), module_id, session_date, topic
Batch	batch_id (PK), course_id, trainer_id, batch_name, start_date, end_date, capacity
Learner	learner_id (PK), learner_name, email, phone
Enrollment	enrollment_id (PK), learner_id, batch_id, enrollment_date, status
Attendance	attendance_id (PK), enrollment_id, session_id, status
Assessment	assessment_id (PK), enrollment_id, assessment_name, score
Fees	fee_id (PK), enrollment_id, amount, payment_date, payment_status
Certificate	certificate_id (PK), enrollment_id, certificate_number (UNIQUE), issue_date, eligibility_status

7.2 Normalization

The database is decomposed into separate relations for independent subjects such as Course, Trainer, Learner, Enrollment and Attendance. Each table stores atomic values and has a primary key, satisfying the practical requirements of First Normal Form (1NF). Non-key descriptive attributes are stored with the entity they describe, reducing repeated data and supporting Second Normal Form (2NF) and Third Normal Form (3NF) in the project design. For example, trainer details are stored once in Trainer rather than repeated in every Batch record, and learner details are stored once in Learner rather than repeated in Attendance, Fees or Assessment.

Implementation note: the current SQL script uses identifier columns to represent logical relationships but does not declare FOREIGN KEY constraints. Adding explicit foreign keys would strengthen referential integrity and prevent inconsistent cross-table references.

8. Data Dictionary

8.1 Course

Column	Data Type	Constraint	Description
course_id	VARCHAR(10)	Primary Key	Unique course identifier
course_name	VARCHAR(100)	-	Name of course
duration_weeks	INT	-	Duration in weeks
fee	DECIMAL(10,2)	-	Course fee

8.2 Module

Column	Data Type	Constraint	Description
module_id	VARCHAR(10)	Primary Key	Unique module identifier
course_id	VARCHAR(10)	Logical reference	Related course ID
module_name	VARCHAR(100)	-	Module name

8.3 Session

Column	Data Type	Constraint	Description
session_id	VARCHAR(10)	Primary Key	Unique session identifier
module_id	VARCHAR(10)	Logical reference	Related module ID
session_date	DATE	-	Date of session
topic	VARCHAR(100)	-	Session topic

8.4 Trainer

Column	Data Type	Constraint	Description
trainer_id	VARCHAR(10)	Primary Key	Unique trainer identifier
trainer_name	VARCHAR(100)	-	Trainer name
specialization	VARCHAR(100)	-	Area of specialization
phone	VARCHAR(15)	-	Contact number

8.5 Batch

Column	Data Type	Constraint	Description
batch_id	VARCHAR(10)	Primary Key	Unique batch identifier
course_id	VARCHAR(10)	Logical reference	Related course ID
trainer_id	VARCHAR(10)	Logical reference	Assigned trainer ID
batch_name	VARCHAR(100)	-	Batch name
start_date	DATE	-	Batch start date
end_date	DATE	-	Batch end date
capacity	INT	-	Maximum capacity

8.6 Learner

Column	Data Type	Constraint	Description
learner_id	VARCHAR(10)	Primary Key	Unique learner identifier
learner_name	VARCHAR(100)	-	Learner name
email	VARCHAR(100)	-	Email address
phone	VARCHAR(15)	-	Contact number

8.7 Enrollment

Column	Data Type	Constraint	Description
enrollment_id	VARCHAR(10)	Primary Key	Unique enrollment identifier
learner_id	VARCHAR(10)	Logical reference	Related learner ID
batch_id	VARCHAR(10)	Logical reference	Related batch ID
enrollment_date	DATE	-	Enrollment date
status	VARCHAR(20)	-	Enrollment status

8.8 Attendance

Column	Data Type	Constraint	Description
attendance_id	VARCHAR(10)	Primary Key	Unique attendance identifier
enrollment_id	VARCHAR(10)	Logical reference	Related enrollment ID
session_id	VARCHAR(10)	Logical reference	Related session ID
status	VARCHAR(10)	-	Present/Absent

8.9 Assessment

Column	Data Type	Constraint	Description
assessment_id	VARCHAR(10)	Primary Key	Unique assessment identifier
enrollment_id	VARCHAR(10)	Logical reference	Related enrollment ID
assessment_name	VARCHAR(100)	-	Assessment title
score	INT	-	Score obtained

8.10 Fees

Column	Data Type	Constraint	Description
fee_id	VARCHAR(10)	Primary Key	Unique feerecord identifier
enrollment_id	VARCHAR(10)	Logical reference	Related enrollment ID
amount	DECIMAL(10,2)	-	Payment amount
payment_date	DATE	-	Date of payment record
payment_status	VARCHAR(20)	-	Paid / Partial / Pending

8.11 Certificate

Column	Data Type	Constraint	Description
certificate_id	VARCHAR(10)	Primary Key	Unique certificate record ID
enrollment_id	VARCHAR(10)	Logical reference	Related enrollment ID
certificate_number	VARCHAR(30)	UNIQUE	Certificate number
issue_date	DATE	-	Certificate issue date
eligibility_status	VARCHAR(20)	-	Eligibility status

9. SQL Commands Used (DDL and DML) with Sample Outputs

9.1 Database Creation

```
CREATE DATABASE IF NOT EXISTS training_management;  
USE training_management;
```

9.2 Sample DDL: Course Table

```
CREATE TABLE Course (  
  course_id VARCHAR(10) PRIMARY KEY,  
  course_name VARCHAR(100),  
  duration_weeks INT,  
  fee DECIMAL(10,2)  
);
```

9.3 Sample DML: Insert Records

```
INSERT INTO Course (course_id, course_name, duration_weeks, fee) VALUES  
( 'C001', 'Python Programming', 8, 12000),  
( 'C002', 'DataAnalytics', 10, 18000);
```

9.4 Sample DML: View Records

```
SELECT * FROM Learner;
```

learner_id	learner_name	email	phone
L001	Ananya Reddy	ananya@gmail.com	9000000001
L002	Rohan Kumar	rohan@gmail.com	9000000002
L003	Meera Nair	meera@gmail.com	9000000003
...	...	...	...
L020	Riya Mehta	riya@gmail.com	9000000020

9.5 Sample DML: Delete Record

```
DELETE FROM Learner WHERE learner_id = 'L021';
```

The complete database_setup.sql file contains the creation and sample insertion commands for all 11 tables.

10. Queries with Outputs

10.1 Number of Learners in the Database

```
SELECT COUNT(*) AS total_learners FROM Learner;
```

total_learners
20

10.2 Courses with Their Batches and Trainers

```
SELECT c.course_name, b.batch_name, t.trainer_name
FROM Batch b
JOIN Course c ON b.course_id = c.course_id
JOIN Trainer t ON b.trainer_id = t.trainer_id;
```

course_name	batch_name	trainer_name
Python Programming	Python-A	Arijun Mehta
Data Analytics	Analytics-A	Priya Sharma
Web Development	WebDev-A	Rahul Verma
Database Management	DBMS-A	Sneha Rao
Machine Learning	ML-A	Karan Singh

10.3 Learners with Pending or Partial Fee Status

```
SELECT l.learner_name, f.payment_status, f.amount
FROM Learner l
JOIN Enrollment e ON l.learner_id = e.learner_id
JOIN Fees f ON e.enrollment_id = f.enrollment_id
WHERE f.payment_status IN ('Pending', 'Partial');
```

learner_name	payment_status	amount
Ishita Sharma	Partial	18000.00
Kavya Singh	Pending	22000.00
Nisha Verma	Pending	10000.00
Diya Joshi	Partial	20000.00
Manav Jain	Pending	24000.00
Harsh Vardhan	Partial	15000.00
Riya Mehta	Pending	28000.00

10.4 Presentation-II Assigned Query

PASTE THE EXACT QUERY GIVEN TO YOU DURING PRESENTATION-II HERE
Then insert its MySQL output screenshot or result table below.

This section is intentionally left as a placeholder because the exact query assigned during Presentation-II was not provided in the available project material.

11. UI Design and Screenshots

The desktop interface is built using Python Tkinter and ttk. It contains four main tabs: Dashboard, View Records, Insert Record and Delete Record. The Dashboard displays record counts for the supported tables. The View tab loads the selected table into a Treeview. The Insert tab dynamically creates input fields for the selected table and sends parameterized INSERT statements to MySQL. The Delete tab deletes a record using the selected table primary key after confirmation.

11.1 Dashboard

Insert a screenshot showing the dashboard and the Connected status.

11.2 View Records

Insert a screenshot showing an existing table such as Learner.

11.3 Insert Record - Before and After

Insert screenshots before insertion and after the new record appears in View Records / MySQL.

11.4 Delete Record - Before and After

Insert screenshots before deletion and after the record disappears from View Records / MySQL.

12. Implementation Details

12.1 Technology Stack

Layer	Technology	Purpose
Front End	Python Tkinter / ttk	Desktop graphical user interface
Application Logic	Python 3	Handles user actions and database calls
DatabaseConnector	mysql-connector-python	Connects Python to MySQL
Database	MySQL	Stores all project records
Development / DB Tool	VS Code and MySQL Workbench	Code editing and database administration
Version Control	GitHub	Stores project files and submission history

12.2 Database Connectivity

```
DB_CONFIG = {  
    "host": "localhost",  
    "user": "root",  
    "password": "<local MySQL password>",  
    "database": "training_management"  
}
```

For security, the actual local MySQL password should not be shown in screenshots or committed to a public GitHub repository. Before uploading the source code, replace the password with an empty value or a placeholder and explain that the user must configure local credentials.

12.3 Key Code Snippets

```
self.connection = mysql.connector.connect(**DB_CONFIG)

cursor.execute(f"SELECT * FROM `{table}`")

cursor.execute(sql, values)
self.connection.commit()

cursor.execute(f"DELETE FROM `{table}` WHERE `{pk}` = %s", (primary_key_value,))
self.connection.commit()
```

The INSERT and DELETE operations use parameter placeholders for values. Table and primary-key names are selected from fixed internal dictionaries in the application rather than accepting arbitrary table names from user input.

13. Testing: Test Cases and Results

ID	Test	Expected Result	Result
TC01	Launch application with correct MySQL settings	UI opens and connection status becomes Connected Existing learner records are displayed	Pass when MySQL is running and credentials are correct
TC02	View Learner table	Success message appears and record is stored	Pass
TC03	Insert a valid new learner, e.g. L021	L021 appears in the table	Pass when a new unique ID is used
TC04	Refresh Learner table after insertion	Confirmation is requested and record is removed	Pass
TC05	Delete learner L021	L021 is no longer displayed	Pass
TC06	Refresh Learner table after deletion	MySQL rejects the duplicate and UI shows an Insert	Pass
TC07	Insert duplicate primary key	Error	Expected validation behavior
TC08	Delete a non-existing primary key	UI shows No record found warning	Expected validation behavior
TC09	Start app with wrong MySQL password	Database Connection Error dialog is displayed	Expected error handling

For the final presentation, TC02 to TC06 should be demonstrated live and the corresponding records can also be checked in MySQL Workbench using SELECT statements.

14. Conclusion and Future Enhancements

14.1 Conclusion

The Training Institute Management System successfully demonstrates the design and implementation of a relational database application using MySQL and Python. The database organizes training-related information across 11 tables, while the Tkinter interface provides the required operations for viewing, inserting and deleting records. The project also demonstrates database connectivity, structured queries, transaction commit/rollback behavior and basic error handling.

14.2 Future Enhancements

- Add explicit FOREIGN KEY constraints for referential integrity.
- Add Update/Edit operation for existing records.
- Add search, filter and sorting functions.
- Add user authentication and role-based access.
- Add attendance percentage and performance reports.
- Add automatic certificate eligibility calculation.
- Add fee balance calculation instead of storing only a payment status.
- Use environment variables or a configuration file for database credentials.
- Develop a web version for multi-user access.

15. References

- MySQL Documentation - SQL syntax and database concepts.
- Python Documentation - Tkinter GUI library.
- MySQL Connector/Python Documentation - Python to MySQL connectivity.

- DBMS course notes and faculty-provided project instructions.

16. Appendix: GitHub Repository Link

Public repository name required by the project instructions: DBMS-Course-Project

GitHub Repository URL: [PASTE YOUR PUBLIC GITHUB REPOSITORY LINK HERE]

Recommended repository folders: Presentation-I, Presentation-II, Presentation-III and Project-Report.

16.1 Files to Keep in Presentation-III

- Presentation-III PPT/PDF
- app.py source code
- requirements.txt
- database_setup.sql or project SQL file
- Screenshots of View, Insert, Delete, before and after states

16.2 Final Checklist Before Submission

- Replace all cover-page placeholders with your details.
- Insert the final ER diagram.
- Insert actual UI screenshots from your working application.
- Add the exact Presentation-II assigned query and its output.
- Paste your GitHub repository URL.
- Remove your real MySQL password from public source code before uploading.
- Export this report to PDF and upload it inside Project-Report.

